

PRAKTIKUM SISTEM OPERASI

MODUL 14

(Scripting)



Oleh:

(Ammar Dzaki Nandana)

(2311104071)

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

UNIVERSITAS TELKOM

2026

MODUL 14

Pada praktikum Modul 14, dilakukan pembelajaran mengenai **Shell Scripting** pada sistem operasi Linux. Shell script merupakan kumpulan perintah Linux yang ditulis dalam sebuah file dan dapat dieksekusi secara otomatis. Dengan shell scripting, pengguna dapat mengotomatisasi pekerjaan yang berulang, melakukan pengolahan data, membuat utilitas sederhana, serta meningkatkan efisiensi administrasi sistem.

Sebelum membuat script, dilakukan persiapan direktori kerja dengan membuat folder khusus untuk menyimpan seluruh file praktikum.

```
mkdir modul14
cd modul14
pwd
```

Perintah `mkdir` digunakan untuk membuat direktori baru bernama *modul14*, kemudian `cd` digunakan untuk masuk ke direktori tersebut, sedangkan `pwd` digunakan untuk memastikan lokasi direktori kerja saat ini.

1. Membuat Script Greeting

Pada percobaan pertama dibuat sebuah script sederhana bernama **greeting.sh** yang berfungsi menampilkan salam, tanggal sistem, serta informasi pengguna yang sedang login.

Langkah Pembuatan

```
nano greeting.sh
```

Source Code

```
#!/bin/bash

echo "Hai ammar"
echo "Hari ini adalah $(date)"
echo "User yang sedang login saat ini adalah: ammar"

who
```

Menjalankan Script

Secara default file script belum dapat dijalankan sehingga perlu diberikan hak akses eksekusi menggunakan perintah:

```
chmod +x greeting.sh
```

Kemudian script dijalankan menggunakan:

```
./greeting.sh
```

Pembahasan

Baris:

```
#!/bin/bash
```

disebut **shebang**, yang berfungsi memberi tahu sistem bahwa script akan dijalankan menggunakan interpreter Bash.

Perintah `echo` digunakan untuk menampilkan teks ke layar terminal. Sementara itu, ekspresi:

```
$(date)
```

akan mengeksekusi perintah `date` dan menampilkan tanggal serta waktu sistem saat script dijalankan.

Perintah:

```
who
```

digunakan untuk menampilkan informasi pengguna yang sedang login pada sistem.

Hasil eksekusi script akan menampilkan salam, tanggal saat ini, dan daftar pengguna aktif pada sistem Linux.

2. Membuat Script Salam Berdasarkan Waktu

Pada percobaan kedua dibuat script yang dapat memberikan salam secara otomatis sesuai waktu saat script dijalankan.

Langkah Pembuatan

```
nano greeting_1.sh
```

Source Code

```
#!/bin/bash
```

```
jam=$(date +%k)
```

```
echo "Sekarang jam $jam"
```

```
if [ "$jam" -ge 5 ] && [ "$jam" -le 10 ]; then  
    echo "Selamat pagi ammar"
```

```
elif [ "$jam" -ge 11 ] && [ "$jam" -le 15 ]; then
```

```

    echo "Selamat siang ammar"

elif [ "$jam" -ge 16 ] && [ "$jam" -le 19 ]; then
    echo "Selamat sore ammar"

else
    echo "Selamat malam ammar"
fi

```

Menjalankan Script

```

chmod +x greeting_1.sh
./greeting_1.sh

```

Pembahasan

Script mengambil informasi jam sistem menggunakan:

```
date +%k
```

Nilai tersebut disimpan ke dalam variabel `jam`.

Struktur kontrol `if`, `elif`, dan `else` digunakan untuk menentukan salam yang sesuai berdasarkan rentang waktu tertentu.

- 05.00–10.59 → Selamat pagi
- 11.00–15.59 → Selamat siang
- 16.00–19.59 → Selamat sore
- Selain itu → Selamat malam

Percobaan ini menunjukkan penggunaan variabel dan percabangan dalam shell scripting.

3. Membuat Countdown Menggunakan Perulangan For

Pada percobaan ketiga dibuat script countdown menggunakan perulangan `for`.

Langkah Pembuatan

```
nano countdown.sh
```

Source Code

```

#!/bin/bash

for ((i=10; i>=1; i--))
do
    echo $i

```

```
done
```

```
echo "GO!"
```

Menjalankan Script

```
chmod +x countdown.sh  
./countdown.sh
```

Pembahasan

Script menggunakan perulangan:

```
for ((i=10; i>=1; i--))
```

yang akan menghitung mundur dari angka 10 hingga 1.

Setelah perulangan selesai, program menampilkan:

```
GO!
```

Percobaan ini menunjukkan penggunaan struktur perulangan `for` untuk melakukan proses yang berulang secara otomatis.

4. Membuat Countdown Menggunakan Input Pengguna

Pada percobaan keempat dibuat countdown yang nilainya ditentukan langsung oleh pengguna.

Langkah Pembuatan

```
nano countdown_1.sh
```

Source Code

```
#!/bin/bash  
  
echo "Masukkan angka:"  
read angka  
  
echo "Mulai countdown!"  
  
while [ $angka -ge 1 ]  
do  
    echo $angka  
    angka=$(( angka-1 ))  
done
```

```
echo "GO!"
```

Menjalankan Script

```
chmod +x countdown_1.sh  
./countdown_1.sh
```

Pembahasan

Perintah:

```
read angka
```

digunakan untuk menerima input dari pengguna.

Selanjutnya, perulangan `while` akan terus berjalan selama nilai variabel `angka` lebih besar atau sama dengan 1.

Setiap iterasi akan mengurangi nilai variabel menggunakan:

```
angka=$(( angka-1 ))
```

hingga mencapai nol dan program menampilkan pesan **GO!**.

Percobaan ini memperlihatkan bagaimana shell script dapat menerima input interaktif dari pengguna.

5. Membuat Countdown Menggunakan Parameter Command Line

Pada percobaan kelima, nilai countdown tidak lagi dimasukkan saat program berjalan, tetapi diberikan melalui parameter ketika script dieksekusi.

Langkah Pembuatan

```
nano countdown_2.sh
```

Source Code

```
#!/bin/bash
```

```
if [ $# -ne 1 ]; then  
    echo "penggunaan: countdown_2.sh initial_value"  
    exit 1  
fi
```

```
angka=$1
```

```
while [ $angka -ge 1 ]
```

```
do
    echo $angka
    angka=$((angka-1))
done

echo "GO!"
```

Menjalankan Script

Memberikan parameter yang benar:

```
chmod +x countdown_2.sh
./countdown_2.sh 5
```

Contoh penggunaan yang salah:

```
./countdown_2.sh
```

Pembahasan

Variabel khusus:

`$#`

menyimpan jumlah parameter yang diberikan saat script dijalankan.

Jika jumlah parameter tidak sama dengan satu, maka program menampilkan pesan penggunaan dan menghentikan eksekusi menggunakan:

```
exit 1
```

Variabel:

`$1`

digunakan untuk mengambil parameter pertama yang diberikan oleh pengguna.

Percobaan ini menunjukkan pemanfaatan command-line argument dalam shell scripting.

6. Menampilkan Isi Direktori Menggunakan Perulangan

Pada percobaan terakhir dibuat script yang digunakan untuk menampilkan seluruh file dan direktori yang berada pada folder aktif.

Langkah Pembuatan

```
nano list_direktori.sh
```

Source Code

```
#!/bin/bash

for file in *
do
    echo $file
done
```

Menjalankan Script

```
chmod +x list_direktori.sh
./list_direktori.sh
```

Pembahasan

Karakter wildcard:

*

merepresentasikan seluruh file dan direktori pada lokasi aktif.

Perulangan:

```
for file in *
```

akan mengambil setiap nama file satu per satu dan menyimpannya ke variabel `file`.

Kemudian perintah:

```
echo $file
```

digunakan untuk menampilkan nama file tersebut ke layar.

Script ini dapat digunakan untuk membuat utilitas sederhana yang memproses seluruh file dalam suatu direktori.

Kesimpulan

Praktikum Modul 14 memberikan pemahaman mengenai dasar-dasar shell scripting pada Linux. Materi yang dipelajari meliputi penggunaan shebang, perintah `echo`, variabel, input pengguna, struktur percabangan `if-elif-else`, perulangan `for` dan `while`, penggunaan parameter command line, serta pemrosesan file dalam direktori. Melalui praktikum ini dapat dipahami bahwa shell script merupakan alat yang sangat berguna untuk mengotomatisasi berbagai pekerjaan administrasi sistem dan meningkatkan efisiensi penggunaan sistem operasi Linux.

